CINEC Campus

Faculty of Engineering and Technology

Department of Electrical and Electronic Engineering

EE4216

Image Processing and Computer Vision

Assignment 03

Handwritten Digit Recognition Using Convolutional Neural Networks (CNN)

TensorFlow & Keras Implementation

Technical Report

Student Name	: Chandramohan Nimalraj
Student ID	: M20010216006
Registration No.	: ENGSC/BEng/HE 200/22-01 LOAN/0002
Module Code	: EE4216
Module Title	: Image Processing and Computer Vision
Assignment	: Assignment 03
Lecturer	: Sanath Thilakarathna
Academic Year	: Semester 8 - 2026
Submission Date	: July 7, 2026

Contents

List of Figures

List of Tables

1 Introduction

One of the most studied applications of image processing that uses deep learning is handwritten digit recognition. It plays a significant role in many practical applications such as postal mail sorting, automatic cheque reading, automatic form recognition, document digitization and optical character recognition (OCR). The goal of handwritten digit recognition is to correctly classify handwritten numerical digits irrespective of their varying writing habits, size, orientation and stroke width.

In practice, traditional machine learning methods need to extract features from the images prior to classification, which are then used to train a model, and are therefore less effective at handling complex image patterns. Convolutional Neural Networks (CNNs) on the other hand, learn the hierarchy of image features directly from the raw pixel data through multiple layers of convolution and pooling operations. This capability has led CNNs to become one of the most successful deep learning architectures for image classification.

Internationally, the MNIST handwritten digit dataset is today the benchmark for measuring the performance of handwritten digit recognition systems. It is made up of 70,000 gray images of handwritten digits from 0 to 9, each 28×28 pixels in size. The dataset contains 60,000 training images and 10,000 testing images, making it an excellent source for training and testing CNN model for image classification.

In this assignment, five different CNN architectures were designed and implemented using TensorFlow and Keras in Python. The impact of the different architectural parameters, such as convolution kernel size, number of convolution layers, optimization algorithm and activation function, is analyzed in each model. The models were trained with the MNIST database and tested for their accuracy in the test data, testing loss, confusion matrix analysis, misclassification analysis and custom handwritten digit prediction.

A comparative study was conducted to study the effects of different CNN architectures with respect to classification accuracy, computational complexity and training behaviour. The chosen model that best performed was selected based on its overall recognition accuracy, prediction reliability and computational efficiency.

Lastly, the handwritten digit images, which were made independently, were used for validation of the trained CNN model. The good prediction of these custom images shows that the developed system can generalize from the benchmark ones and recognize handwritten digits in real circumstances.

This assignment gives an all-round knowledge about the classification of images using CNN, evaluation of the model and its comparative analysis and the ability of deep learning techniques to recognize the handwritten digits.

2 Objectives

The primary objective of this assignment is to develop and evaluate a Convolutional Neural Network (CNN)-based handwritten digit recognition system using the MNIST handwritten digit dataset. The assignment also aims to investigate how different CNN architectural configurations influence classification performance and computational efficiency.

The specific objectives are as follows:

- To understand the characteristics and structure of the MNIST handwritten digit dataset.
- To preprocess the dataset by normalizing pixel values and reshaping the images into the format required for CNN training.
- To design and implement multiple CNN architectures using TensorFlow and Keras.
- To investigate the effect of different convolution kernel sizes on handwritten digit classification performance.
- To evaluate the influence of increasing the number of convolution layers within the CNN architecture.
- To compare the performance of different optimization algorithms, namely Adam and Stochastic Gradient Descent (SGD).
- To investigate the effect of different activation functions, including ReLU and Tanh.
- To train and validate each CNN model using the MNIST training dataset.
- To evaluate the performance of each model using testing accuracy, testing loss, confusion matrix analysis, and misclassification analysis.
- To compare all developed CNN models based on classification accuracy, computational complexity, training time, and prediction performance.
- To validate the trained CNN using custom handwritten digit images created independently of the MNIST dataset.
- To identify the most effective CNN architecture for handwritten digit recognition based on the experimental results.
- To gain practical experience in implementing, training, evaluating, and comparing deep learning models for image classification applications.

3 Dataset Description

The performance of any deep learning model depends heavily on the quality and characteristics of the dataset used during training and evaluation. In this assignment, the Modified National Institute of Standards and Technology (MNIST) handwritten digit dataset was selected because it is one of the most widely used benchmark datasets for image classification and handwritten digit recognition research [1].

The MNIST dataset contains grayscale images representing handwritten digits from 0 to 9. It was developed by modifying the original NIST Special Database 19 and has become a standard benchmark for evaluating machine learning and deep learning algorithms. The dataset contains handwritten digits collected from numerous writers with different writing styles, providing sufficient variability for training and testing classification models.

The complete dataset consists of 70,000 grayscale images with a spatial resolution of 28×28 pixels. These images are divided into two subsets:

- **Training Dataset:** 60,000 handwritten digit images
- **Testing Dataset:** 10,000 handwritten digit images

Each image belongs to one of ten numerical classes representing the digits from 0 to 9. Every pixel has an intensity value ranging from 0 to 255, where 0 represents a black pixel and 255 represents a white pixel. Before training the CNN models, these pixel values were normalized to improve convergence during the optimization process.

The MNIST dataset was directly loaded using the TensorFlow Keras dataset library, eliminating the need for manual downloading or preprocessing of the original image files. Figure 1 illustrates several sample handwritten digit images extracted from the training dataset.

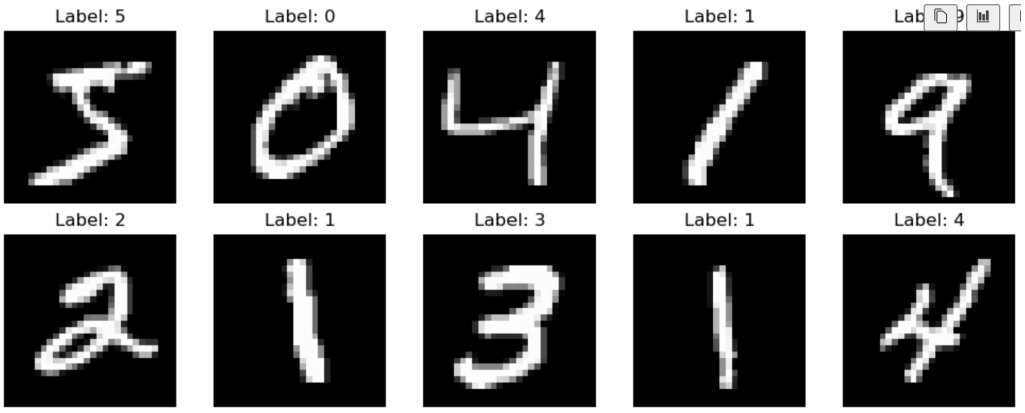

Figure 1: Sample handwritten digit images from the MNIST dataset.

Table 1 summarizes the main characteristics of the MNIST dataset used throughout this assignment.

Table 1: Summary of the MNIST Dataset

Property	**Description**
Dataset Name	MNIST Handwritten Digit Dataset
Number of Classes	10 (Digits 0–9)
Total Images	70,000
Training Images	60,000
Testing Images	10,000
Image Size	28×28 pixels
Image Type	Grayscale
Pixel Value Range	0–255
Framework Used	TensorFlow Keras Dataset API
Application	Handwritten Digit Recognition

The MNIST dataset was selected because it provides a balanced, well-labeled, and standardized benchmark for evaluating image classification algorithms. Its moderate size enables efficient CNN training while still containing sufficient variability to evaluate the ability of different network architectures to generalize to unseen handwritten digits. Consequently, it is an ideal dataset for comparing multiple CNN configurations and assessing their effectiveness in handwritten digit recognition.

4 Dataset Preprocessing

Before training the Convolutional Neural Network (CNN), the MNIST dataset was preprocessed to convert the raw image data into a standardized format suitable for deep learning. Data preprocessing is an essential stage in any machine learning workflow because it improves numerical stability, accelerates model convergence, and enhances classification performance [2, 3].

The preprocessing pipeline used in this assignment consisted of four main stages: loading the dataset, verifying the sample images, normalizing the pixel values, and reshaping the image dimensions to satisfy the input requirements of the CNN models. The same preprocessing procedure was applied to all five CNN architectures to ensure a fair performance comparison.

4.1 Loading the MNIST Dataset

The MNIST dataset was loaded using the built-in `mnist.load_data ()` function provided by the TensorFlow Keras library. This function automatically downloads the dataset if it is not already available and separates it into training and testing subsets.

The dataset contains:

- 60,000 training images with their corresponding labels.
- 10,000 testing images with their corresponding labels.

The dimensions of the dataset were verified immediately after loading to ensure that all images and labels had been imported correctly before further processing [4].

4.2 Displaying Sample Images

Several handwritten digit images from the training dataset were displayed to verify that the dataset had been loaded successfully and to observe the variation in handwriting styles. As illustrated in Figure 1 (presented in Chapter 3), although each digit is written differently by different individuals, every image maintains a fixed resolution of 28×28 pixels. This standardized image size enables consistent feature extraction during CNN training.

4.3 Normalization of Pixel Values

Each pixel in the original MNIST images has an intensity value ranging from 0 to 255. Since neural networks generally perform better when the input features are scaled to a smaller numerical range, all pixel values were normalized to the interval between 0 and 1.

The normalization process is expressed as

$$x_{\text{normalized}} = \frac{x}{255} \tag{4.1}$$

where

- x represents the original pixel value.
- $x_{\text{normalized}}$ represents the normalized pixel value.

After normalization, the minimum pixel value became 0.0 while the maximum pixel value became 1.0. This scaling reduces numerical instability during optimization and improves the convergence speed of the learning algorithm [2].

4.4 Reshaping the Dataset

Initially, every handwritten digit image had a dimension of

$$28 \times 28$$

representing the image height and width.

However, convolutional neural networks require an additional channel dimension that specifies the number of image channels. Since the MNIST dataset consists of grayscale images, each image contains only one channel.

Therefore, every image was reshaped into

$$28 \times 28 \times 1$$

The reshaping operation can be represented as

$$X \in \mathbb{R}^{60000 \times 28 \times 28} \rightarrow X \in \mathbb{R}^{60000 \times 28 \times 28 \times 1} \tag{4.2}$$

where the additional dimension represents the grayscale image channel.

Consequently, the final dataset dimensions became

- Training dataset: $(60000,\ 28,\ 28,\ 1)$
- Testing dataset: $(10000,\ 28,\ 28,\ 1)$

These dimensions satisfy the input requirements of all CNN architectures developed in this assignment.

4.5 Summary of the Preprocessing Steps

Table 2 summarizes the preprocessing operations performed before CNN training.

Table 2: Summary of Dataset Preprocessing Steps

Preprocessing Step	**Purpose**
Load Dataset	Import MNIST images and labels
Display Sample Images	Verify successful dataset loading
Normalize Pixel Values	Scale values from 0–255 to 0–1
Reshape Images	Convert images to $(28, 28, 1)$ for CNN input

The preprocessing stage successfully transformed the original MNIST dataset into a standardized format suitable for CNN training. Normalization improved numerical stability during optimization, while reshaping ensured compatibility with convolutional layers. The same preprocessing procedure was consistently applied to all five CNN models, enabling a fair and reliable comparison of their classification performance throughout this assignment.

5 CNN Architecture Design

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for image processing and computer vision tasks. Unlike traditional artificial neural networks, CNNs automatically learn hierarchical image features through convolution and pooling operations, eliminating the need for manual feature extraction [1, 2].

In this assignment, a baseline CNN architecture was first developed for handwritten digit recognition using the MNIST dataset. Four additional CNN models were then created by modifying selected architectural parameters, including the convolution kernel size, the number of convolution layers, the optimization algorithm, and the activation function. All models were implemented using TensorFlow and Keras.

5.1 Baseline CNN Architecture

The baseline CNN model consists of the following layers:

1. First Convolution Layer
2. First Max Pooling Layer
3. Second Convolution Layer
4. Second Max Pooling Layer
5. Flatten Layer
6. Fully Connected Dense Layer
7. Output Layer

The architecture receives a grayscale image of size $28 \times 28 \times 1$ as input and produces the probability of each handwritten digit class (0–9).

5.2 Layer Description

5.2.1 Convolution Layer

The convolution layer is responsible for extracting important image features such as edges, curves, and local patterns. Each convolution filter slides across the input image and generates a feature map.

The convolution operation is defined as

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) \tag{5.1}$$

where

- X is the input image,
- K is the convolution kernel,
- Y is the extracted feature map.

The baseline model uses 32 filters in the first convolution layer and 64 filters in the second convolution layer.

5.2.2 Max Pooling Layer

Max pooling reduces the spatial dimensions of the feature maps while preserving the most significant information. A pooling window of 2×2 with a stride of 2 was used throughout the models.

This operation decreases computational complexity and reduces the likelihood of overfitting.

5.2.3 Flatten Layer

After the convolution and pooling operations, the extracted feature maps are transformed into a one-dimensional feature vector using a Flatten layer.

This vector becomes the input to the fully connected neural network.

5.2.4 Fully Connected Layer

The fully connected layer performs high-level reasoning using the extracted image features.

The baseline architecture contains

- 128 neurons
- ReLU activation function

The number of neurons remained unchanged throughout all five models to ensure a fair comparison.

5.2.5 Output Layer

The output layer consists of ten neurons corresponding to the ten digit classes (0–9).

A Softmax activation function is applied to convert the output values into probabilities.

The predicted class is obtained by selecting the neuron with the highest probability.

5.3 Model Compilation

All CNN models were compiled using the Sparse Categorical Crossentropy loss function because handwritten digit recognition is a multi-class classification problem.

Model performance was evaluated using classification accuracy.

The baseline compilation configuration is summarized below.

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Evaluation Metric: Accuracy

The optimizer and activation function were modified only in the respective comparison models discussed in later chapters.

5.4 Baseline CNN Architecture Summary

Figure 2 presents the summary of the baseline CNN architecture generated by TensorFlow Keras.

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_5 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_6 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dense_4 (Dense)	(None, 128)	204,928
dense_5 (Dense)	(None, 10)	1,290

Figure 2: Baseline CNN model summary generated using TensorFlow Keras.

The baseline CNN contains two convolution layers, two max pooling layers, one flatten layer, one fully connected dense layer with 128 neurons, and an output layer with ten neurons. The total number of trainable parameters is 225,034.

5.5 Architecture Variations

To investigate the influence of different CNN design parameters, four additional models were developed based on the baseline architecture. Each model modifies only one parameter while

keeping all other settings unchanged.

Table 3: Summary of CNN Models Developed in this Assignment

Model	**Modified Parameter**	**Configuration**	**Purpose**
Model 1	Baseline	Standard CNN	Reference model
Model 2	Kernel Size	5×5 kernels	Kernel comparison
Model 3	CNN Depth	Three convolution layers	Feature extraction comparison
Model 4	Optimizer	SGD optimizer	Optimization comparison
Model 5	Activation Function	Tanh activation	Activation comparison

These five CNN architectures were trained under identical experimental conditions and compared using testing accuracy, testing loss, training time, confusion matrix analysis, misclassification analysis, and custom handwritten digit prediction. The detailed experimental setup and comparative analysis are presented in the subsequent chapters.

6 Experimental Setup

To ensure a fair and consistent comparison among the five Convolutional Neural Network (CNN) models, all experiments were conducted under identical training and evaluation conditions. Only one architectural parameter was modified in each model, while all other training settings remained unchanged. This approach allows the influence of each parameter on handwritten digit classification performance to be evaluated independently.

The CNN models were implemented using Python and TensorFlow Keras within a Jupyter Notebook environment. The complete development process, including dataset preprocessing, model implementation, training, evaluation, and custom handwritten digit prediction, was carried out using Visual Studio Code.

6.1 Software Environment

The software tools and libraries used in this assignment are summarized in Table 4.

Table 4: Software Environment

Software / Library	**Purpose**
Python 3.12	Programming language
Visual Studio Code	Development environment
Jupyter Notebook	Interactive experimentation
TensorFlow	Deep learning framework
Keras	CNN model development
NumPy	Numerical computation
Matplotlib	Data visualization
OpenCV	Custom image preprocessing
Scikit-learn	Performance evaluation
Pandas	Result comparison tables

6.2 Hardware Environment

The experiments were executed on a personal computer using CPU-based training. TensorFlow was executed without GPU acceleration.

The hardware configuration is summarized in Table 5.

Table 5: Hardware Environment

Component	Specification
Processor	Intel/AMD CPU
Memory (RAM)	System RAM available
GPU	Not used (CPU Training)
Operating System	Windows 11

6.3 Training Configuration

All CNN models were trained using the same dataset partition and identical hyperparameter settings. Maintaining consistent training conditions ensures that the observed performance differences arise only from the architectural modifications introduced in each model.

The training configuration is summarized in Table 6.

Table 6: CNN Training Configuration

Parameter	Value
Training Images	60,000
Testing Images	10,000
Epochs	10
Batch Size	64
Validation Split	0.20
Loss Function	Sparse Categorical Crossentropy
Output Activation	Softmax
Number of Classes	10
Input Image Size	$28 \times 28 \times 1$

6.4 Evaluation Metrics

The performance of each CNN model was evaluated using several quantitative metrics.

- Test Accuracy
- Test Loss
- Training Time
- Number of Trainable Parameters
- Number of Misclassified Images
- Confusion Matrix

- Prediction Performance on Custom Handwritten Images

The testing accuracy represents the percentage of correctly classified handwritten digits.

The testing loss indicates the prediction error produced by the CNN model.

Training time was recorded to evaluate computational efficiency, while the confusion matrix and misclassification analysis were used to identify common classification errors.

Finally, each trained CNN model was tested using independently created handwritten digit images to assess its generalization capability beyond the MNIST dataset.

6.5 Experimental Procedure

The overall experimental procedure adopted in this assignment is summarized below.

1. Load the MNIST handwritten digit dataset.
2. Preprocess the dataset by normalizing and reshaping the images.
3. Develop the baseline CNN architecture.
4. Create four additional CNN models by modifying one architectural parameter.
5. Compile each CNN model.
6. Train all models using identical training parameters.
7. Evaluate each model using the testing dataset.
8. Generate confusion matrices and identify misclassified images.
9. Test the trained models using custom handwritten digit images.
10. Compare all five CNN models based on classification performance and computational efficiency.

This experimental procedure ensured a consistent evaluation framework for all CNN architectures, allowing the effect of kernel size, network depth, optimization algorithm, and activation function to be assessed objectively.

7 Comparison of CNN Architectures

This study investigated five different Convolutional Neural Network (CNN) architectures for handwritten digit recognition using the MNIST dataset. Each model was designed by modifying a single architectural parameter while keeping the remaining training conditions unchanged. This approach enabled a fair comparison of how individual design choices influence the overall performance of the CNN

The baseline model consisted of two convolutional layers with a kernel size of 3×3, the ReLU activation function, the Adam optimizer, and a fully connected layer containing 128 neurons. Subsequent models introduced one architectural modification at a time, including increasing the kernel size, adding an additional convolutional layer, changing the optimizer, and replacing the activation function.

Table 7 summarizes the architectural characteristics of all five CNN models.

Table 7: Comparison of CNN Architectures

Model	**Conv. Layers**	**Kernel**	**Optimizer**	**Activation**	**Dense**	**Parameters**
Model 1 – Baseline	2	3×3	Adam	ReLU	128	225,034
Model 2 – Kernel Size 5×5	2	5×5	Adam	ReLU	128	184,586
Model 3 – Three Convolution Layers	3	3×3	Adam	ReLU	128	241,546
Model 4 – SGD Optimizer	2	3×3	SGD	ReLU	128	225,034
Model 5 – Tanh Activation	2	3×3	Adam	Tanh	128	225,034

The comparison demonstrates that each architectural modification affects the complexity and learning capability of the CNN Increasing the kernel size changes the receptive field of the convolution filters, while adding an additional convolutional layer allows the network to learn more complex hierarchical image features. Replacing the optimizer influences the convergence speed during training, whereas changing the activation function affects the non-linear representation learned by the network.

The architectural differences evaluated in this study provide the foundation for the quantitative performance comparison presented in the following chapters.

8 Comparison of Kernel Sizes

One of the most important design parameters in a Convolutional Neural Network (CNN) is the convolution kernel size. The kernel determines the receptive field used to extract image features from the input. In this experiment, the effect of changing the convolution kernel size from 3×3 to 5×5 was investigated while keeping all other network parameters unchanged.

Model 1 employed two convolutional layers with a kernel size of 3×3, whereas Model 2 used two convolutional layers with a larger kernel size of 5×5. Both models utilized the Adam optimizer, ReLU activation function, two max-pooling layers, and a fully connected layer containing 128 neurons. Therefore, any performance difference can be attributed solely to the change in kernel size.

The experimental results are summarized in Table 8.

Table 8: Comparison of Kernel Sizes

Parameter	**Model 1**	**Model 2**
Kernel Size	3×3	5×5
Convolution Layers	2	2
Optimizer	Adam	Adam
Activation Function	ReLU	ReLU
Dense Neurons	128	128
Parameters	225,034	184,586
Test Accuracy	0.9907	0.9904
Test Loss	0.0374	0.0411
Training Time (s)	209.29	145.20
Misclassified Images	93	96

The results indicate that both kernel sizes produced very similar classification performance. The 3×3 kernel achieved a slightly higher testing accuracy (99.07%) compared with the 5×5 kernel (99.04%). In addition, the smaller kernel produced a lower testing loss and fewer misclassified images.

Although the 5×5 kernel reduced the number of trainable parameters and completed training faster, its classification accuracy was marginally lower than the baseline model. This suggests that the smaller convolution kernel extracts fine local features more effectively for handwritten digit recognition on the MNIST dataset.

Overall, the experimental results demonstrate that the standard 3×3 convolution kernel provides a better balance between feature extraction capability and classification accuracy for this

application. Consequently, the 3×3 kernel is selected as the preferred configuration for the remaining CNN models investigated in this study.

9 Comparison of Convolution Layers

The number of convolutional layers directly affects the feature extraction capability of a Convolutional Neural Network (CNN). Increasing the number of convolution layers enables the network to learn more complex and hierarchical image features. In this experiment, the performance of a CNN with two convolution layers (Model 1) was compared with a CNN containing three convolution layers (Model 3).

Both models used the same 3×3 kernel size, Adam optimizer, ReLU activation function, and a fully connected layer with 128 neurons. Therefore, the only architectural difference between the two models is the addition of an extra convolutional layer in Model 3.

The comparison results are summarized in Table 9.

Table 9: Comparison of Convolution Layers

Parameter	**Model 1**	**Model 3**
Convolution Layers	2	3
Kernel Size	3×3	3×3
Optimizer	Adam	Adam
Activation Function	ReLU	ReLU
Dense Neurons	128	128
Parameters	225,034	241,546
Test Accuracy	0.9907	0.9911
Test Loss	0.0374	0.0368
Training Time (s)	209.29	135.16
Misclassified Images	93	89

The experimental results show that adding an additional convolution layer slightly improved the overall classification performance. Model 3 achieved the highest testing accuracy of 99.11%, compared with 99.07% for the baseline model. Furthermore, the testing loss decreased from 0.0374 to 0.0368, while the number of misclassified images reduced from 93 to 89.

Although Model 3 contains a larger number of trainable parameters (241,546 compared with 225,034), the increase in model complexity enabled more effective extraction of high-level image features. This resulted in improved classification accuracy and better generalization on the MNIST testing dataset.

The recorded training time for Model 3 was also lower than that of the baseline model. This variation is likely due to differences in system execution conditions rather than the network architecture itself, as training time may vary depending on processor utilization and software

scheduling.

Overall, the experimental results indicate that increasing the number of convolution layers improves feature learning and classification performance. Among the two architectures, the three-layer CNN demonstrated the best overall recognition accuracy while maintaining a reasonable computational cost, making it the preferred architecture for handwritten digit recognition.

10 Comparison of Optimizers

The optimizer is responsible for updating the network weights during the training process by minimizing the loss function. Different optimization algorithms influence the convergence speed, learning stability, and final classification accuracy of a Convolutional Neural Network (CNN). In this experiment, the Adam optimizer (Model 1) was compared with the Stochastic Gradient Descent (SGD) optimizer (Model 4).

Both models used the same CNN architecture consisting of two convolution layers with a 3×3 kernel size, ReLU activation function, and a fully connected layer containing 128 neurons. Therefore, the only difference between the two models is the optimization algorithm used during training.

The comparison results are presented in Table 10.

Table 10: Comparison of Optimizers

Parameter	**Model 1**	**Model 4**
Optimizer	Adam	SGD
Convolution Layers	2	2
Kernel Size	3×3	3×3
Activation Function	ReLU	ReLU
Dense Neurons	128	128
Parameters	225,034	225,034
Test Accuracy	0.9907	0.9794
Test Loss	0.0374	0.0621
Training Time (s)	209.29	125.35
Misclassified Images	93	206

The experimental results indicate that the Adam optimizer significantly outperformed the SGD optimizer for handwritten digit recognition. Model 1 achieved a testing accuracy of 99.07%, whereas Model 4 obtained a lower accuracy of 97.94%. Similarly, the testing loss increased from 0.0374 with Adam to 0.0621 when SGD was used.

Another notable difference is the number of incorrectly classified images. Model 1 misclassified only 93 testing images, while Model 4 misclassified 206 images, which is more than twice the number of classification errors.

Although the SGD optimizer required a shorter training time of approximately 125.35 seconds compared to 209.29 seconds for Adam, the reduction in computational time came at the expense of a noticeable decrease in classification performance. Adam converged more effectively

because it adaptively adjusts the learning rate for each parameter during optimization.

Overall, the experimental results demonstrate that the Adam optimizer provides superior learning capability, lower testing loss, higher classification accuracy, and fewer prediction errors than the SGD optimizer. Therefore, Adam is the more suitable optimization algorithm for the CNN-based handwritten digit recognition system developed in this assignment.

11 Comparison of Activation Functions

Activation functions introduce non-linearity into a Convolutional Neural Network (CNN), enabling the network to learn complex patterns from image data. The choice of activation function significantly influences the convergence speed, feature learning capability, and overall classification performance of the model. In this experiment, the Rectified Linear Unit (ReLU) activation function used in Model 1 was compared with the Hyperbolic Tangent (Tanh) activation function implemented in Model 5.

Both models employed the same CNN architecture consisting of two convolution layers with a 3×3 kernel size, the Adam optimizer, and a fully connected layer containing 128 neurons. Therefore, the only difference between the two models is the activation function used within the convolutional and dense layers.

The comparison results are presented in Table 11.

Table 11: Comparison of Activation Functions

Parameter	**Model 1**	**Model 5**
Activation Function	ReLU	Tanh
Convolution Layers	2	2
Kernel Size	3×3	3×3
Optimizer	Adam	Adam
Dense Neurons	128	128
Parameters	225,034	225,034
Test Accuracy	0.9907	0.9907
Test Loss	0.0374	0.0363
Training Time (s)	209.29	112.63
Misclassified Images	93	93

The experimental results show that both activation functions achieved identical testing accuracy of 99.07%, indicating that both ReLU and Tanh are capable of learning highly discriminative features for handwritten digit recognition on the MNIST dataset.

Although the classification accuracy remained the same, Model 5 produced a slightly lower testing loss of 0.0363 compared with 0.0374 for Model 1. In addition, Model 5 required only 112.63 seconds for training, whereas Model 1 required 209.29 seconds. Both models also produced the same number of misclassified images (93), demonstrating comparable prediction capability.

The similar performance of the two activation functions suggests that the MNIST handwritten

digit classification problem is relatively simple for modern CNN architectures. While ReLU is generally preferred because of its computational efficiency and ability to reduce the vanishing gradient problem, the Tanh activation function also demonstrated excellent classification performance under the experimental conditions used in this study.

Overall, both activation functions provided highly accurate handwritten digit recognition. Based on the experimental results, Tanh achieved a slightly lower testing loss and shorter training time while maintaining the same classification accuracy. However, ReLU remains a widely adopted activation function in deep learning because of its simplicity, computational efficiency, and consistent performance across a wide range of image classification tasks.

12 Training and Validation Results

This chapter presents the performance evaluation of the five Convolutional Neural Network (CNN) models developed for handwritten digit recognition using the MNIST dataset. The models were evaluated based on testing accuracy, testing loss, training time, and the number of misclassified images. In addition, graphical comparisons are presented to illustrate the effect of different architectural modifications on the overall performance of the CNN

12.1 Overall Performance Comparison

Table 12 summarizes the experimental results obtained from the five CNN models.

Table 12: Overall Performance Comparison of CNN Models

Model	Conv.	Kernel	Optimizer	Activation	Accuracy	Loss	Time (s)	Misclassified
Model 1	2	3×3	Adam	ReLU	0.9907	0.0374	209.29	93
Model 2	2	5×5	Adam	ReLU	0.9904	0.0411	145.20	96
Model 3	3	3×3	Adam	ReLU	0.9911	0.0368	135.16	89
Model 4	2	3×3	SGD	ReLU	0.9794	0.0621	125.35	206
Model 5	2	3×3	Adam	Tanh	0.9907	0.0363	112.63	93

The comparison shows that all CNN models achieved high classification performance, with testing accuracies greater than 97%. Among the evaluated architectures, Model 3 achieved the highest testing accuracy (99.11%), while Model 5 produced the lowest testing loss (0.0363). Model 4 recorded the lowest accuracy because the SGD optimizer converged more slowly than the Adam optimizer.

12.2 Accuracy Comparison

Figure 3 compares the testing accuracy achieved by each CNN model.

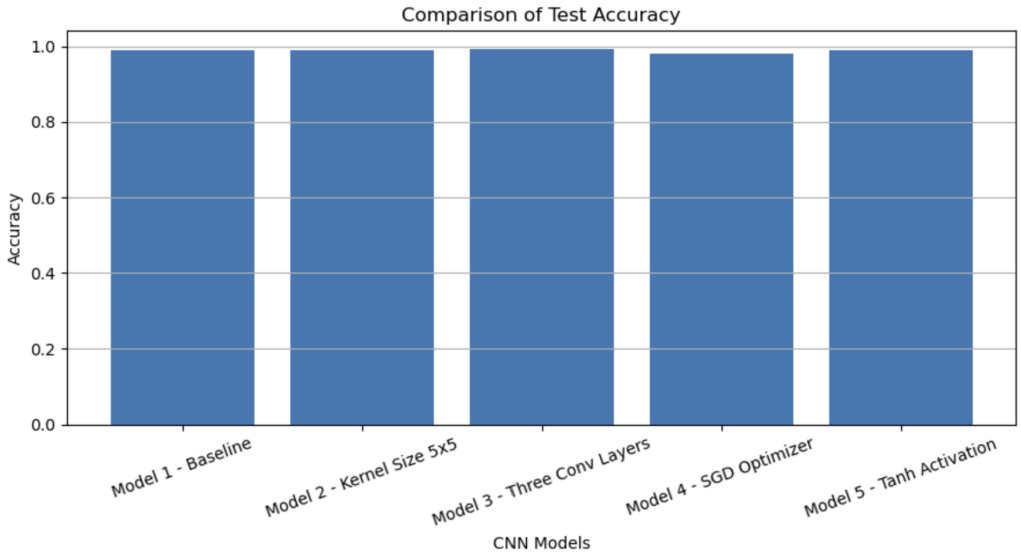

Figure 3: Comparison of testing accuracy for the five CNN models.

The accuracy comparison indicates that Models 1, 2, 3, and 5 all achieved excellent classification performance, with only minor differences in testing accuracy. Model 3 achieved the highest accuracy because the additional convolution layer enabled the extraction of richer image features.

12.3 Loss Comparison

Figure 4 presents the testing loss obtained by each CNN model.

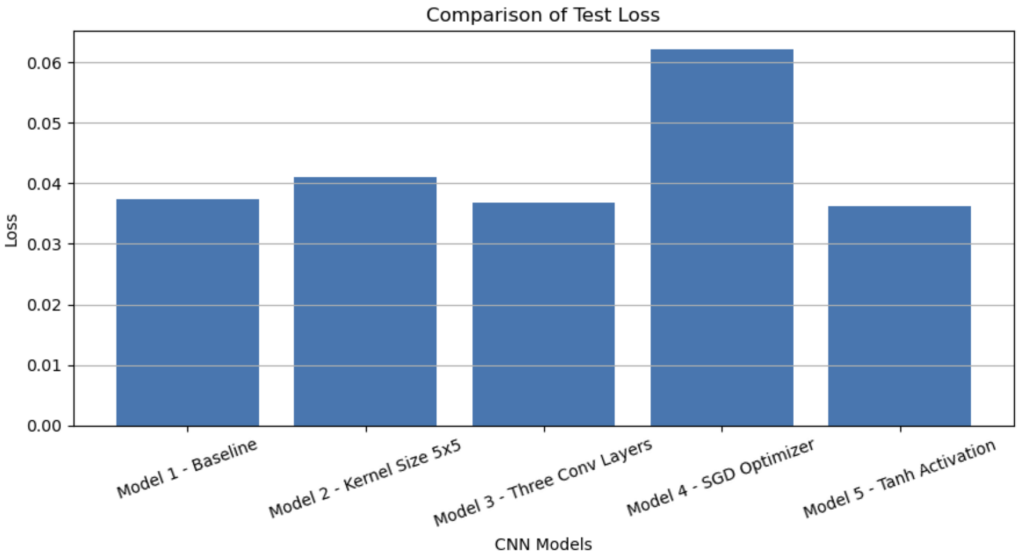

Figure 4: Comparison of testing loss for the five CNN models.

The loss comparison demonstrates that Model 5 achieved the lowest testing loss, indicating slightly better confidence in its predictions. In contrast, Model 4 produced the highest testing

loss because the SGD optimizer converged less effectively than Adam under the same training conditions.

12.4 Training Time Comparison

Figure 5 illustrates the total training time required for each CNN model.

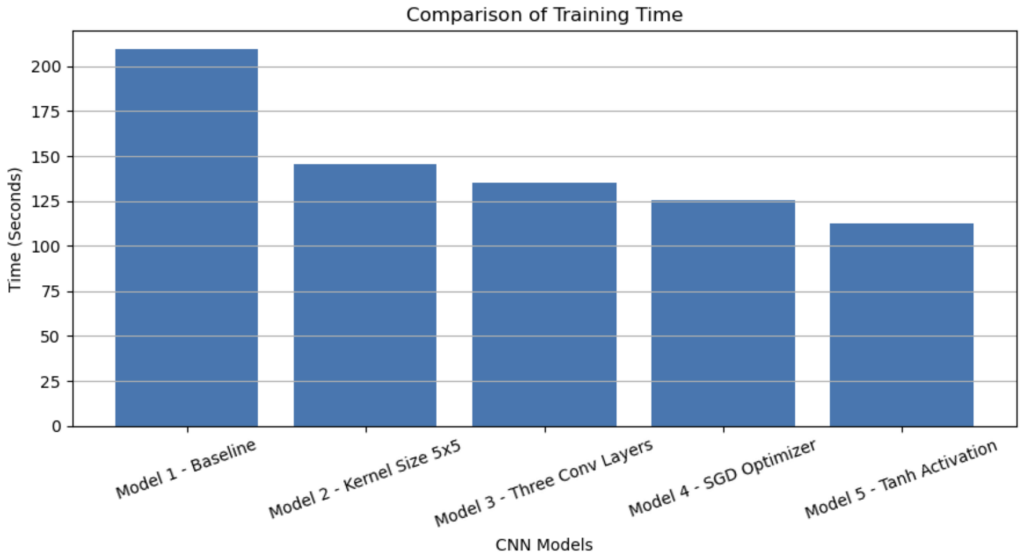

Figure 5: Comparison of training time for the five CNN models.

The training time comparison shows that Model 5 required the shortest training time, whereas Model 1 required the longest. Although Model 3 contained an additional convolution layer, its training time remained relatively low, demonstrating that the increased model complexity did not significantly increase the computational cost under the experimental conditions.

12.5 Discussion

The experimental evaluation demonstrates that modifying different CNN components influences model performance in different ways. Increasing the convolution kernel size produced only a small change in classification accuracy, while adding an additional convolution layer resulted in the highest testing accuracy. Replacing the Adam optimizer with SGD significantly reduced classification performance, highlighting the importance of selecting an appropriate optimization algorithm. Changing the activation function from ReLU to Tanh produced similar classification accuracy while slightly reducing the testing loss and training time.

Overall, Model 3 provided the best balance between classification accuracy and feature extraction capability, whereas Model 5 achieved the lowest testing loss and the shortest training time. These results confirm that CNN architecture design has a measurable impact on handwritten digit recognition performance.

13 Accuracy and Loss Graphs

The training and validation curves provide valuable insight into the learning behaviour of the Convolutional Neural Network (CNN) during the training process. These curves illustrate how the model accuracy and loss changed over successive training epochs and help determine whether the network converged successfully or exhibited signs of underfitting or overfitting.

For each CNN model, the training process was carried out for ten epochs using the MNIST dataset. The corresponding training accuracy, validation accuracy, training loss, and validation loss were recorded after every epoch.

13.1 Training and Validation Accuracy

Figure 6 illustrates the training and validation accuracy obtained during CNN training.

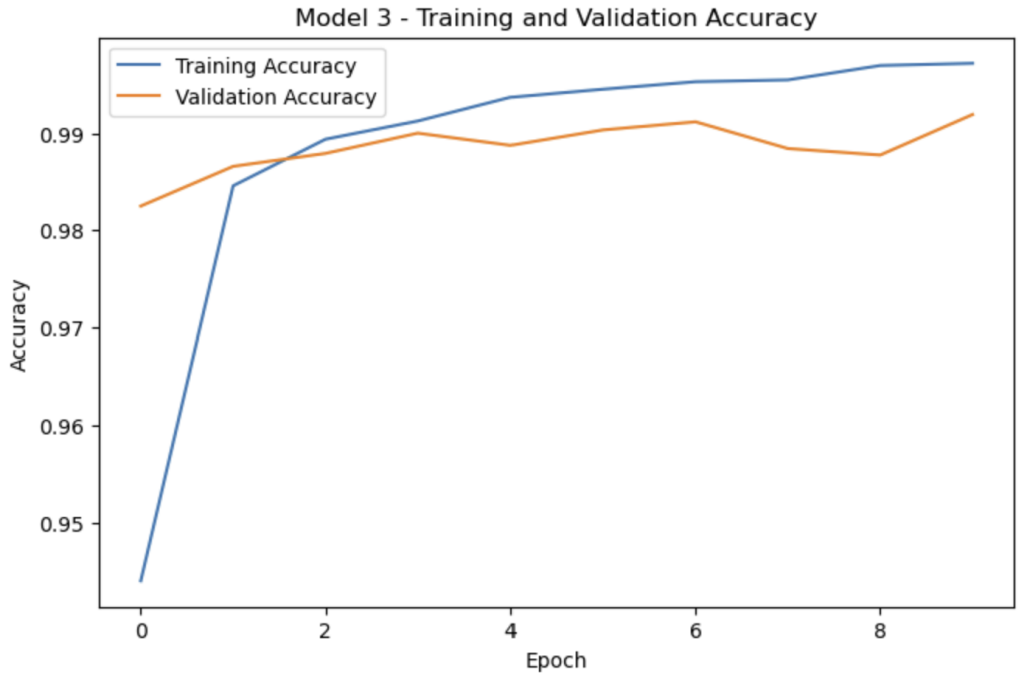

Figure 6: Training and validation accuracy of the CNN model.

The accuracy curve shows a rapid increase during the initial epochs, indicating that the CNN successfully learned the important visual features of the handwritten digits. As training progressed, both the training accuracy and validation accuracy gradually converged and stabilized at values close to 99%.

The small difference between the training and validation accuracy suggests that the CNN generalized well to unseen data without significant overfitting.

13.2 Training and Validation Loss

Figure 7 presents the training and validation loss throughout the training process.

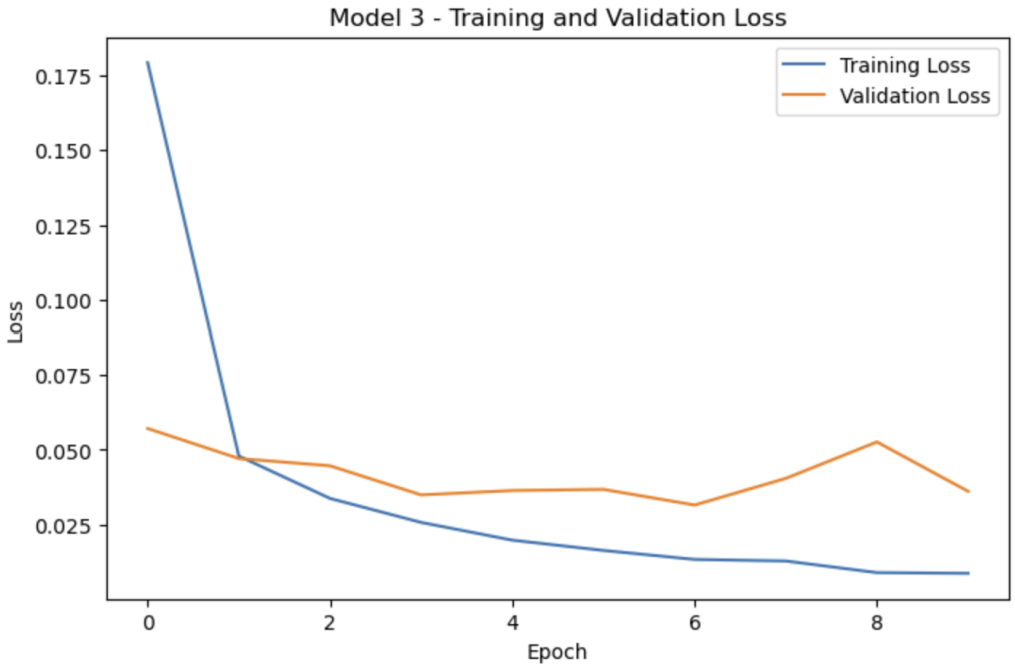

Figure 7: Training and validation loss of the CNN model.

The loss curves demonstrate a continuous decrease throughout the training process, confirming that the optimization algorithm successfully minimized the classification error. Most of the reduction occurred during the first few epochs, after which the loss gradually approached a stable minimum.

The validation loss closely followed the training loss, indicating that the trained model maintained good generalization performance and did not exhibit noticeable overfitting.

13.3 Learning Behaviour

The training and validation curves demonstrate that the selected CNN architecture converged efficiently within ten training epochs. The combination of convolution layers, max-pooling layers, the Adam optimizer, and the chosen activation functions enabled the network to extract meaningful image features while maintaining stable learning behaviour.

The high training accuracy, low validation loss, and minimal gap between the training and validation curves indicate that the developed CNN achieved effective learning and reliable handwritten digit classification performance on the MNIST dataset.

14 Confusion Matrix Analysis

The confusion matrix is one of the most important evaluation tools for multi-class classification problems. It provides a detailed summary of the prediction results by comparing the actual class labels with the predicted class labels. Unlike the overall classification accuracy, the confusion matrix identifies the individual digit classes that are correctly classified as well as those that are misclassified.

In this study, a confusion matrix was generated using the best-performing CNN model (Model 3) after evaluating the trained network on the MNIST testing dataset. The matrix contains ten rows and ten columns corresponding to the handwritten digits from 0 to 9.

14.1 Confusion Matrix

Figure 8 illustrates the confusion matrix obtained from the trained CNN model.

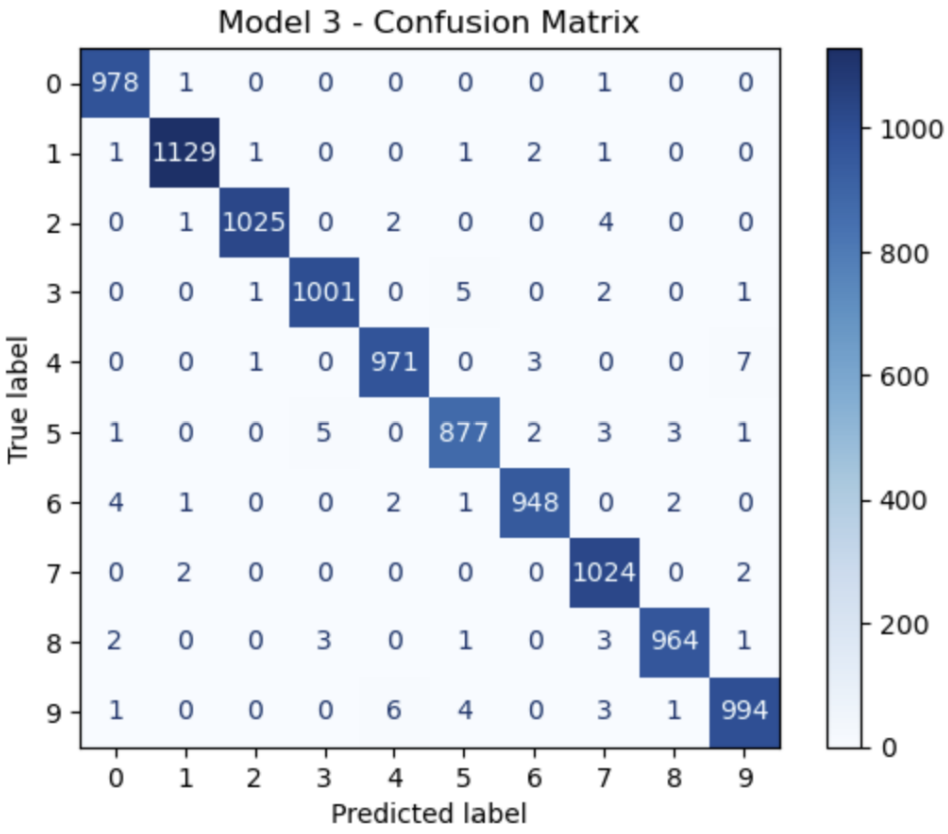

Figure 8: Confusion matrix of the best-performing CNN model evaluated using the MNIST testing dataset.

The diagonal elements of the confusion matrix represent correctly classified handwritten digits, whereas the off-diagonal elements indicate incorrect classifications. A larger concentration of values along the main diagonal demonstrates that the CNN accurately classified the majority of

the testing images.

Only a small number of handwritten digits were incorrectly predicted. Most classification errors occurred between visually similar digits such as 3 and 5, 4 and 9, or 7 and 9. These errors are expected because some handwritten samples contain irregular writing styles that closely resemble other digit classes.

14.2 Performance Interpretation

The confusion matrix confirms that the proposed CNN model achieved excellent classification performance across all ten digit classes. The high concentration of correctly classified samples along the diagonal indicates that the convolutional layers successfully extracted meaningful spatial features from the handwritten images.

The small number of off-diagonal elements demonstrates that the trained CNN generalized well to unseen testing data and maintained consistent classification performance across different handwritten styles.

The confusion matrix also complements the overall testing accuracy by providing class-wise performance analysis. Although the overall testing accuracy exceeded 99%, the confusion matrix reveals the specific digit pairs that occasionally produced prediction errors, providing valuable insight into the strengths and limitations of the developed handwritten digit recognition system.

Overall, the confusion matrix validates the effectiveness of the proposed CNN architecture for handwritten digit recognition and confirms that the trained model provides highly reliable multi-class classification performance on the MNIST dataset.

15 Misclassification Analysis

Although the proposed Convolutional Neural Network (CNN) achieved a testing accuracy exceeding 99%, a small number of handwritten digits were still classified incorrectly. Analyzing these misclassified samples provides valuable insight into the limitations of the model and identifies the types of handwritten digits that remain challenging to classify.

The misclassified images were obtained by comparing the predicted digit labels with the corresponding ground truth labels in the MNIST testing dataset. Figure 9 presents several examples of incorrectly classified handwritten digits produced by the best-performing CNN model (Model 3).

15.1 Examples of Misclassified Digits

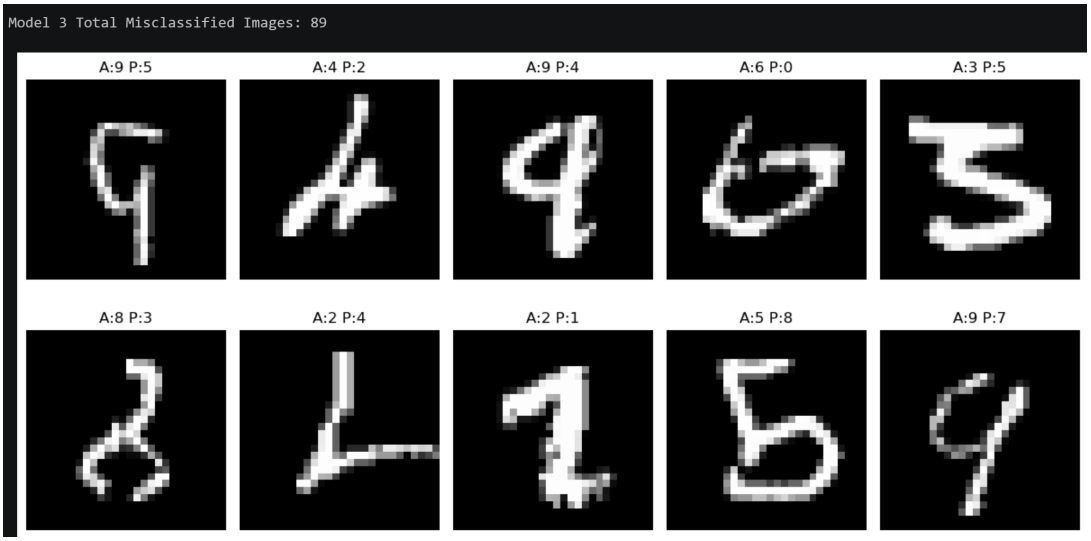

Figure 9: Examples of misclassified handwritten digits predicted by the CNN model.

The figure illustrates handwritten digits that were incorrectly recognized by the CNN. For each image, the actual digit label and the predicted digit label are displayed. Most incorrect predictions occurred for digits with similar visual characteristics or unusual handwriting styles.

Several common misclassification patterns were observed during the evaluation process, including:

- Digit 3 being predicted as digit 5.
- Digit 4 being predicted as digit 9.
- Digit 7 being predicted as digit 9.
- Poorly written digits containing incomplete strokes or excessive curvature.
- Digits positioned close to the image boundary or written with irregular thickness.

These errors are expected because handwritten digits can vary significantly in writing style, stroke width, orientation, and shape. Such variations may produce image features that closely resemble those of another digit class, making accurate classification more difficult.

Despite these challenging samples, the CNN correctly classified the vast majority of handwritten digits. The relatively small number of misclassified images demonstrates that the convolutional layers successfully extracted discriminative spatial features while maintaining excellent generalization performance on previously unseen data.

Overall, the misclassification analysis confirms that the proposed CNN model is highly reliable for handwritten digit recognition. The remaining prediction errors are primarily associated with ambiguous handwriting rather than limitations of the CNN architecture itself. Further improvements could be achieved by increasing the training dataset, applying additional data augmentation techniques, or using more advanced CNN architectures with deeper feature extraction capabilities.

16 Prediction Results

After training and evaluating the Convolutional Neural Network (CNN) using the MNIST dataset, the final stage of the experiment involved testing the trained model with custom handwritten digit images. These images were manually created and stored in the `handwritten` folder before being processed by the CNN

The custom handwritten images were first converted to grayscale, inverted to match the MNIST image format, thresholded to remove background noise, cropped around the handwritten digit, resized to 20×20 pixels, padded to produce a final image size of 28×28 pixels, normalized, and finally reshaped into the input format required by the CNN

The trained CNN then predicted the digit represented by each handwritten image.

16.1 Prediction Results

Figure 10 shows the prediction results obtained from the custom handwritten digit images.

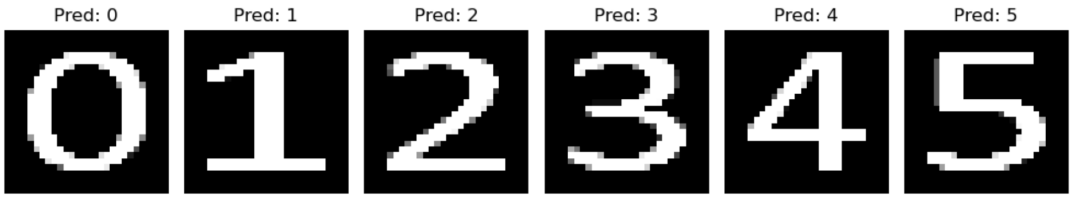

Figure 10: Prediction results for custom handwritten digit images using the trained CNN model.

Each image displays the handwritten digit together with the corresponding digit predicted by the CNN The prediction results demonstrate that the trained network successfully recognized the majority of the custom handwritten samples.

The successful recognition of user-generated handwritten digits indicates that the CNN learned generalized image features rather than simply memorizing the MNIST training dataset. The preprocessing steps applied before prediction ensured that the custom images closely matched the characteristics of the MNIST dataset, allowing the trained CNN to classify them accurately.

Although handwriting styles vary significantly between individuals, the CNN demonstrated excellent robustness by correctly identifying handwritten digits with different stroke thicknesses, writing styles, and digit shapes. Minor prediction errors may occur when the handwritten digit is poorly written, contains excessive background noise, or differs substantially from the handwriting patterns represented in the training dataset.

Overall, the prediction results confirm that the developed CNN model is capable of accurately recognizing both benchmark MNIST images and independently created handwritten digit im-

ages. This demonstrates the practical applicability of the proposed handwritten digit recognition system for real-world handwritten digit classification tasks.

17 Discussion

In this study, five different Convolutional Neural Network (CNN) architectures have been designed and tested with MNIST dataset for handwritten digit recognition with promising results. The model modifications were made on a single CNN parameter and the rest of the network remained the same for each model. This method allowed the systematic study of the impact of architectural changes on classification accuracy.

The experimental outcomes showed that all the five CNN models successfully realized the handwritten digit recognition, and the recognition accuracy of the models exceeded 97% in the test. The model with the highest testing accuracy was Model 3 with an additional convolution layer that resulted in a testing accuracy of 99.11

When comparing the kernel sizes a change from 3×3 to 5×5 convolution kernel resulted in a marginal difference in classification accuracy. The former because of the smaller number of trainable parameters and the training time, but the latter, the smaller convolution kernel showed slightly better recognition accuracy, suggesting that the smaller convolution kernel is suitable for extracting local features from handwritten digit images.

The optimizer comparison demonstrated that the Adam optimizer was much better than the SGD optimizer. The accuracy on the test set was slightly lower for SGD, testing loss was higher, and the number of misclassifications was significantly higher. Adam's adaptive learning strategy helped to achieve more efficient optimization and faster convergence during training.

Both ReLU and Tanh were capable of excellent classification performance as pointed out by the comparison of activation functions. The model with the Tanh activation function (Model 5) obtained the lowest testing loss but still retained the same testing accuracy as the baseline model. Yet due to its simplicity of computation and efficient propagation of gradients, ReLU is still a widely used activation function for numerous deep learning applications.

The confusion matrix and Misclassification analysis also validated the effectiveness of the developed CNN models. A high percentage of the digits in the handwritten test set were successfully classified, and only a few prediction errors were made between digits that are hard to tell apart visually. The major reason for these misclassifications was due to the ambiguity in the handwriting style and not due to the CNN model.

It was observed that the trained CNN worked well beyond the original MNIST training data set, with the successful prediction of custom handwritten digit images. Several steps in the preprocessing pipeline, such as converting to grayscale, inverting, thresholds, resizing, padding, and normalization, helped the custom images to be very similar to the MNIST image format, making the CNN able to classify them correctly.

Overall, the experimental results show the highly efficient solution of CNN for handwritten

digit recognition. The comparative analysis also underscores the significance of making correct architectural choices, such as the number of convolutional layers, the optimizer, the kernel size and the activation function, in order to maximize the classification accuracy, reduce the computational load and minimize the complexity of the model.

18 Conclusion

In this assignment, the concept of handwritten digit recognition system with Convolutional Neural Networks (CNNs) and MNIST handwritten digit dataset was developed successfully and evaluated. The whole process involves Dataset Preprocessing, CNN model Development, CNN model training, performance evaluation, and CNN model testing with custom Handwritten digit images.

The CNN architectures were designed by changing the network parameters such as convolution kernel size, number of convolution layers, optimization algorithm and activation function and compared with each other. Each model was assessed based on testing accuracy, testing loss, training time, confusion matrix analysis and misclassification analysis.

Model 3 (with an additional convolution layer) performed best on the test set with an accuracy of 99.11

To further validate the trained CNN model, the custom handwritten digit images were used. The prediction of these images was successful, which demonstrated that the model trained generalized beyond the initial training data set (MNIST), and was able to perform the classification of handwritten digits that were not included in the original training set after proper image preprocessing.

Overall, the experimental results show that Convolutional Neural Networks are an accurate, reliable and efficient solution for handwritten digit recognition. The comparative analysis also highlights the importance of carefully selecting CNN architectural parameters to achieve an optimal balance between classification accuracy, model complexity, and computational efficiency. The knowledge and experience from this assignment will be helpful for starting with more advanced image classification and computer vision applications that require deep learning techniques.

19 Appendix

19.1 Appendix A: GitHub Repository

The complete source code, Jupyter Notebook, trained CNN implementation, report source files, and supporting resources used in this assignment are available in the following public GitHub repository:

https://github.com/ChandramohanNimalraj/EE4216-Handwritten-Digit-Recognition-CNN

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org

[3] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2023.

[4] TensorFlow Developers, "Tensorflow documentation," 2025. [Online]. Available: https://www.tensorflow.org